ISLAMIC UNIVERSITY

Department of Information and Communication Technology

LABORATORY MANUAL

ICT-4202

Artificial Intelligence and Machine Learning Laboratory

Prepared by

Prof. Dr. Md. Zahidul Islam

Professor

Department of Information and Communication Technology

Islamic University, Kushtia, Bangladesh

Maximum number of experiments: 10

Prepared for instructional use based on the supplied ICT-4202 syllabus | August 2026

Preface

This laboratory manual is designed for ICT-4202: Artificial Intelligence and Machine Learning Laboratory. It converts the syllabus topics into ten structured, hands-on experiments that move from intelligent agents and symbolic AI to search, WEKA, supervised learning, and unsupervised learning. Closely related syllabus sub-experiments are grouped so that the total number of experiments does not exceed ten.

Each experiment contains learning objectives, essential theory, prerequisites, a step-by-step procedure, complete solution code or tool instructions, expected results, interpretation, post-lab exercises, viva questions with concise answers, and a submission checklist. The manual is intended to support both guided laboratory sessions and independent practice.

Important: The experiment numbering in this manual is a teaching design based on the supplied syllabus. It does not alter any officially approved university assessment rule or course ordinance.

Course Information

Field	Information
Course code	ICT-4202
Course title	Artificial Intelligence and Machine Learning Laboratory
Credit	1.5
Contact hours	39
Full marks	100
Prerequisite	ICT-4201
Primary tools used in this manual	Python 3, Jupyter/IDLE/VS Code (any one), WEKA Explorer

Course Learning Outcomes (CLOs)

CLO	Outcome used for this manual
CLO1	Define and explain Artificial Intelligence, rationality, autonomy, and different types of AI agents.
CLO2	Analyze AI search algorithms, including uninformed, informed, heuristic, and related problem-solving techniques.
CLO3	Implement fundamentals of knowledge representation and simple knowledge-based systems using logic-oriented representations.
CLO4	Apply fundamental machine-learning algorithms to solve analytical problems.

CLO mapping note: The weekly table in the supplied syllabus contains some later entries marked CLO2-3, while the CLO definitions identify machine learning under CLO4. This manual maps ML experiments to CLO4 according to the wording of the CLO definitions.

Experiment Map

No.	Experiment	CLO
1	Intelligent Agents and Simple Reflex Agent	CLO1
2	Knowledge-Based Agents and Propositional Logic	CLO3
3	First-Order Logic for Knowledge Representation	CLO3
4	Conventional Search: BFS, DFS, Bidirectional and Greedy Best-First	CLO2
5	Depth-Limited Search and Iterative Deepening	CLO2
6	Heuristic Functions and A* Search	CLO2
7	Introduction to WEKA for AI/ML Experiments	CLO4 (tool preparation)
8	Supervised Learning: Linear Regression	CLO4
9	Supervised Learning: k-NN and Decision Tree Classification	CLO4
10	Unsupervised Learning: K-means Clustering	CLO4

Laboratory Rules and Good Practice

- Attend the laboratory with the required software installed and complete any pre-lab reading before the session.
- Create a separate folder for each experiment and keep source code, datasets, screenshots, and the final report together.
- Run code yourself. Do not submit code copied from another student without understanding and attribution.
- For machine-learning experiments, record the dataset, train/test split, random seed, parameters, and evaluation metric so results are reproducible.
- Do not change test data to improve accuracy. Report unexpected results and explain them instead of hiding them.

- For WEKA, save important result-buffer output or screenshots and include them in the lab record when required by the instructor.
- Use meaningful variable names and comments. A program that runs but cannot be explained in viva is not considered a complete laboratory outcome.

Software Setup

1. Install Python 3.10 or later from a trusted distribution. Confirm installation by running: python --version
2. Create a working folder, for example: ICT4202_Lab.
3. Install the packages used in the ML experiments:

```
python -m pip install numpy pandas matplotlib scikit-learn
```

4. Install WEKA and a compatible Java runtime if WEKA is not already available on the laboratory computers.
5. Test Python by running:

```
print("ICT-4202 AI/ML Lab environment is ready")
```

6. For Experiments 8-10, using Jupyter Notebook is recommended but not compulsory; .py files are fully acceptable.

Standard Lab Report Format

7. Experiment number and title
8. Date, student name, roll/registration number, and group
9. Objectives
10. Short theory / key concepts
11. Algorithm or procedure
12. Source code / WEKA configuration
13. Output / observations
14. Result interpretation and discussion
15. Answers to assigned post-lab tasks
16. Conclusion
17. Viva preparation / instructor remarks and signature

Suggested Per-Experiment Record Rubric (Instructor Use)

Component	Suggested weight
Preparation / pre-lab understanding	10%
Procedure and correct execution	25%
Code / model configuration quality	25%
Observation and interpretation	20%
Post-lab tasks and viva	20%

EXPERIMENT 1

Intelligent Agents and a Simple Reflex Agent

Item	Details
CLO mapping	CLO1
Recommended time	3 laboratory hours
Submission	Source code / screenshots where applicable, observation, discussion, answers to post-lab tasks

Objectives

- Identify agent, environment, percept, action, sensor, and actuator concepts.
- Explain PEAS (Performance measure, Environment, Actuators, Sensors).
- Implement and trace a simple reflex vacuum-cleaner agent in Python.

Essential Theory

An intelligent agent perceives its environment through sensors and acts through actuators. A rational agent selects an action expected to maximize its performance measure given the percept sequence and built-in knowledge. A simple reflex agent chooses an action using condition-action rules based only on the current percept.

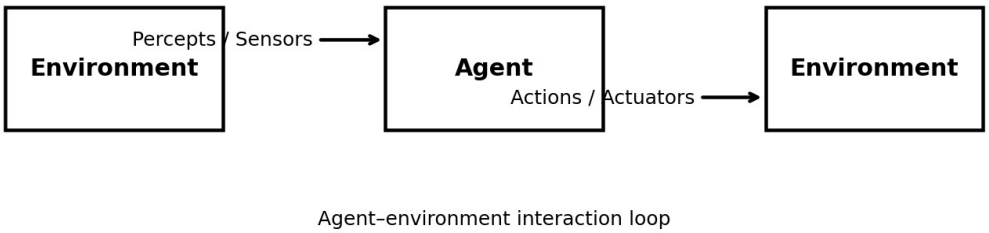


Figure 1. Agent-environment interaction.

Problem Statement

Consider a two-room vacuum world with locations A and B. Each room can be Clean or Dirty. The agent can perform Suck, Left, or Right. Implement a simple reflex agent that cleans the current room if dirty; otherwise it moves to the other room.

PEAS Description

PEAS element	Vacuum-world design
Performance	Keep both rooms clean; avoid unnecessary actions.
Environment	Two rooms A and B; each room Clean/Dirty.
Actuators	Suck, move Left, move Right.

PEAS element	Vacuum-world design
Sensors	Current location and status of current room.

Step-by-Step Procedure

18. Create a new file named exp1_agent.py.
19. Define a function that receives current location and room status.
20. If status is Dirty, return Suck.
21. If the room is Clean, move Right from A and Left from B.
22. Represent the environment using a Python dictionary.
23. Run the agent for several steps and print each percept-action pair.
24. Verify that both rooms become clean.

Complete Solution

```
def simple_reflex_vacuum_agent(location, status):
    if status == "Dirty":
        return "Suck"
    return "Right" if location == "A" else "Left"

environment = {"A": "Dirty", "B": "Dirty"}
location = "A"

for step in range(6):
    status = environment[location]
    action = simple_reflex_vacuum_agent(location, status)
    print(step + 1, location, status, "->", action)

    if action == "Suck":
        environment[location] = "Clean"
    elif action == "Right":
        location = "B"
    elif action == "Left":
        location = "A"

print("Final environment:", environment)
```

Expected / Sample Output: 1 A Dirty -> Suck
2 A Clean -> Right
3 B Dirty -> Suck
4 B Clean -> Left
...
Final environment: {'A': 'Clean', 'B': 'Clean'}

Solution Explanation

The agent is reactive: it does not remember earlier percepts. The condition-action rule Dirty => Suck is always given priority. When the room is clean, the agent moves to the other location. After both rooms are clean, the simple reflex policy continues moving because it has no internal state or stopping goal; this limitation motivates model-based and goal-based agents.

Post-Lab Tasks

- Modify the environment so that only room B starts dirty. Record the trace.

- Add an action NoOp so the agent stops when both rooms are clean. What additional information must the agent use?
- Write a PEAS description for an autonomous university delivery robot.

Viva Questions with Short Answers

Question	Short answer
What is a percept?	The information an agent receives from the environment at a particular time.
What is a rational agent?	An agent that selects actions expected to maximize its performance measure given available information.
Why is the program a simple reflex agent?	Its action depends only on the current percept using condition-action rules.
What is a limitation of a simple reflex agent?	It cannot use percept history or an internal model to handle partially observable situations.

EXPERIMENT 2

Knowledge-Based Agents and Propositional Logic

Item	Details
CLO mapping	CLO3
Recommended time	3 laboratory hours
Submission	Source code / screenshots where applicable, observation, discussion, answers to post-lab tasks

Objectives

- Represent facts and rules in a small knowledge base.
- Explain implication and rule-based inference.
- Implement forward chaining for propositional rules.

Essential Theory

A knowledge-based agent stores explicit knowledge and uses inference to derive new information. In propositional logic, statements are either true or false. A rule such as Rain -> WetGround means that if Rain is known to be true, WetGround can be inferred. Forward chaining repeatedly applies rules whose premises are satisfied until no new fact can be derived.

Problem Statement

Build a knowledge base with the facts and rules: Rain; Rain -> WetGround; WetGround -> Slippery; Slippery -> DriveSlowly. Use forward chaining to infer all reachable conclusions.

Step-by-Step Procedure

25. Create a set containing the initial fact Rain.
26. Represent each rule as a pair: set of premises and one conclusion.

27. Copy initial facts into an inferred set.
28. Scan every rule. If all premises are in inferred and the conclusion is new, add it.
29. Repeat scanning until a complete pass adds no new fact.
30. Print the final inferred knowledge and the inference trace.

Complete Solution

```
facts = {"Rain"}
rules = [
    ({"Rain"}, "WetGround"),
    ({"WetGround"}, "Slippery"),
    ({"Slippery"}, "DriveSlowly"),
]

def forward_chain(facts, rules):
    inferred = set(facts)
    changed = True
    trace = []

    while changed:
        changed = False
        for premises, conclusion in rules:
            if premises.issubset(inferred) and conclusion not in inferred:
                inferred.add(conclusion)
                trace.append((premises, conclusion))
                changed = True
    return inferred, trace

knowledge, trace = forward_chain(facts, rules)
print("Inferred knowledge:", sorted(knowledge))
for premises, conclusion in trace:
    print(set(premises), "=>", conclusion)
```

Expected / Sample Output: Inferred knowledge: ['DriveSlowly', 'Rain', 'Slippery', 'WetGround']
{'Rain'} => WetGround
{'WetGround'} => Slippery
{'Slippery'} => DriveSlowly

Truth-Table Check

For implication P -> Q, the implication is false only when P is true and Q is false.

P	Q	P -> Q
False	False	True
False	True	True
True	False	False
True	True	True

Interpretation

The knowledge base begins with one asserted fact. Because each derived conclusion satisfies the premise of the next rule, the inference chain reaches DriveSlowly. Forward chaining is data-driven: it starts from known facts and derives consequences.

Post-Lab Tasks

- Add the rule {Rain, Cold} -> WearJacket. Test once with Cold absent and once with Cold present.
- Add an unrelated rule and explain why it does not fire.
- Explain the difference between a fact and a rule in this implementation.

Viva Questions with Short Answers

Question	Short answer
What is a knowledge base?	A collection of represented facts and rules used by an agent for reasoning.
What is forward chaining?	A data-driven inference method that repeatedly applies rules to known facts.
When is P -> Q false?	Only when P is true and Q is false.
What does premises.issubset(inferred) test?	Whether every premise of a rule is already known to be true.

EXPERIMENT 3

First-Order Logic for Knowledge Representation

Item	Details
CLO mapping	CLO3
Recommended time	3 laboratory hours
Submission	Source code / screenshots where applicable, observation, discussion, answers to post-lab tasks

Objectives

- Distinguish predicates, constants, variables, and relations.
- Represent binary relations as facts.
- Derive a simple relation corresponding to a first-order rule.

Essential Theory

First-order logic (FOL) extends propositional logic by representing objects and relationships. For example Parent(Rahim, Karim) is a predicate with two constants. A rule for grandparents can be written informally as: Parent(x, y) AND Parent(y, z) -> Grandparent(x, z). This experiment implements the same relation using Python tuples and nested matching.

Knowledge Base

FOL-style fact	Meaning
Parent(Rahim, Karim)	Rahim is a parent of Karim.
Parent(Karim, Nadia)	Karim is a parent of Nadia.

FOL-style fact	Meaning
Parent(Rahim, Mina)	Rahim is a parent of Mina.
Parent(Mina, Rafi)	Mina is a parent of Rafi.

Step-by-Step Procedure

31. Represent each Parent fact as a tuple (parent, child) inside a set.
32. Write children_of(person) to query all tuples whose first element equals person.
33. Implement the grandparent rule by finding two Parent tuples where the child of the first equals the parent of the second.
34. Add (first parent, second child) to the inferred grandparent set.
35. Print queries and inferred results.

Complete Solution

```
parents = {
   ("Rahim", "Karim"),
   ("Karim", "Nadia"),
   ("Rahim", "Mina"),
   ("Mina", "Rafi"),
}

def children_of(person):
   return sorted(child for parent, child in parents if parent == person)

def infer_grandparents():
   result = set()
   for p1, c1 in parents:
      for p2, c2 in parents:
         if c1 == p2:
            result.add((p1, c2))
   return sorted(result)

print("Children of Rahim:", children_of("Rahim"))
print("Grandparent relations:", infer_grandparents())
```

Expected / Sample Output: Children of Rahim: ['Karim', 'Mina']
Grandparent relations: [('Rahim', 'Nadia'), ('Rahim', 'Rafi')]

Solution Explanation

The nested loop performs a simple variable-binding operation. When c1 and p2 match, the program has found values for x, y, and z that satisfy Parent(x,y) and Parent(y,z). It then derives Grandparent(x,z). This is a computational illustration of relational reasoning, not a full FOL theorem prover.

Post-Lab Tasks

- Add more people and verify that the inferred grandparent relation expands correctly.
- Implement sibling(x,y): two different people are siblings if they share at least one parent.
- Explain why first-order logic is more expressive than propositional logic.

Viva Questions with Short Answers

Question	Short answer
What is a predicate?	A symbol representing a property or relation, such as Parent(x,y).
What are constants?	Names of specific objects, such as Rahim and Karim.
What is a variable?	A placeholder that can be bound to objects, such as x, y, or z.
Is this program a complete theorem prover?	No. It demonstrates one rule-specific inference pattern.

EXPERIMENT 4

Conventional Search: BFS, DFS, Bidirectional and Greedy Best-First

Item	Details
CLO mapping	CLO2
Recommended time	4 laboratory hours
Submission	Source code / screenshots where applicable, observation, discussion, answers to post-lab tasks

Objectives

- Implement breadth-first search (BFS).
- Implement depth-first search (DFS).
- Understand bidirectional search.
- Implement Greedy Best-First Search using a heuristic.
- Compare completeness and path behavior.

Graph used in Experiment 4

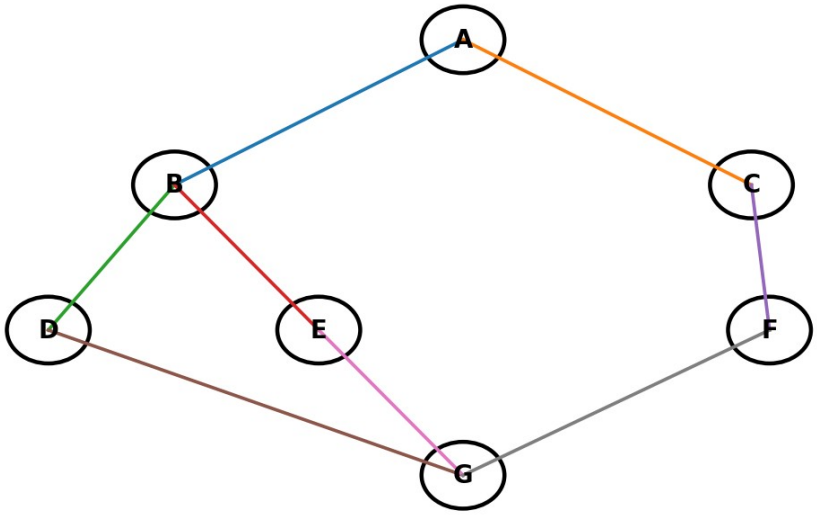


Figure 2. Common graph used for comparing search strategies.

Search Problem

Find a path from A to G in the graph shown above. Adjacency order is kept fixed so that the trace is reproducible.

Step-by-Step Procedure

36. Store the graph as an adjacency-list dictionary.
37. For BFS, store complete paths in a FIFO queue and expand the shallowest path first.
38. For DFS, use a LIFO stack and expand one branch deeply before backtracking.
39. For bidirectional search, grow frontiers from both start and goal until they meet.
40. For Greedy Best-First Search, assign a heuristic h(n) estimating closeness to G and expand the lowest-h node first.
41. Run all algorithms from A to G and compare paths.

Complete Solution

```
from collections import deque
import heapq

graph = {
    "A": ["B", "C"],
    "B": ["A", "D", "E"],
    "C": ["A", "F"],
    "D": ["B", "G"],
    "E": ["B", "G"],
    "F": ["C", "G"],
    "G": ["D", "E", "F"]
}

def bfs(graph, start, goal):
    q = deque([[start]])
    visited = {start}
    while q:
        path = q.popleft()
        node = path[-1]
        if node == goal:
            return path
        for nbr in graph[node]:
            if nbr not in visited:
                visited.add(nbr)
                q.append(path + [nbr])

def dfs(graph, start, goal):
    stack = [(start, [start])]
    visited = set()
    while stack:
        node, path = stack.pop()
        if node == goal:
            return path
        if node not in visited:
            visited.add(node)
            for nbr in reversed(graph[node]):
                if nbr not in visited:
                    stack.append((nbr, path + [nbr]))

def bidirectional(graph, start, goal):
    if start == goal:
        return [start]
    qs, qg = deque([start]), deque([goal])
    ps, pg = {start:[start]}, {goal:[goal]}
    vs, vg = {start}, {goal}
```

```
    while qs and qg:
        for _ in range(len(qs)):
            node = qs.popleft()
            for nbr in graph[node]:
                if nbr not in vs:
                    vs.add(nbr); ps[nbr] = ps[node] + [nbr]; qs.append(nbr)
                    if nbr in vg:
                        return ps[nbr] + list(reversed(pg[nbr][:-1]))
        for _ in range(len(qg)):
            node = qg.popleft()
            for nbr in graph[node]:
                if nbr not in vg:
                    vg.add(nbr); pg[nbr] = pg[node] + [nbr]; qg.append(nbr)
                    if nbr in vs:
                        return ps[nbr] + list(reversed(pg[nbr][:-1]))

heuristic = {"A":6, "B":4, "C":4, "D":2, "E":2, "F":1, "G":0}

def greedy_best_first(graph, start, goal, h):
    pq = [(h[start], start, [start])]
    visited = set()
    while pq:
        _, node, path = heapq.heappop(pq)
        if node == goal:
            return path
        if node in visited:
            continue
        visited.add(node)
        for nbr in graph[node]:
            if nbr not in visited:
                heapq.heappush(pq, (h[nbr], nbr, path + [nbr]))

print("BFS:", bfs(graph, "A", "G"))
print("DFS:", dfs(graph, "A", "G"))
print("Bidirectional:", bidirectional(graph, "A", "G"))
print("Greedy:", greedy_best_first(graph, "A", "G", heuristic))
```

Expected / Sample Output: BFS: ['A', 'B', 'D', 'G']
DFS: ['A', 'B', 'D', 'G']
Bidirectional: ['A', 'B', 'D', 'G']
Greedy: ['A', 'B', 'D', 'G']

Comparison

Algorithm	Main data structure	Key behavior
BFS	FIFO queue	Complete for finite branching; shortest path in number of edges when all step costs are equal.
DFS	LIFO stack	Memory efficient, but may follow a deep/unhelpful branch and is not generally optimal.
Bidirectional	Two frontiers	Can reduce effective search depth when reverse search is possible.
Greedy best-first	Priority queue by h(n)	Uses heuristic guidance; can be fast but is not generally optimal.

Post-Lab Tasks

- Change adjacency order and observe whether DFS returns a different path.
- Add an extra node H and compare the number of expanded nodes.
- Use a deliberately poor heuristic and show how Greedy Best-First can be misled.

Viva Questions with Short Answers

Question	Short answer
Why does BFS use a queue?	A queue expands nodes in increasing depth order.
Why can DFS use less memory?	It mainly stores the current search frontier along deep branches rather than all nodes at a depth.
What does bidirectional search require?	A way to search from the goal side and detect intersection of the frontiers.
Does Greedy Best-First guarantee the lowest-cost path?	No. It selects using h(n) alone, not accumulated path cost.

EXPERIMENT 5

Depth-Limited Search and Iterative Deepening

Item	Details
CLO mapping	CLO2
Recommended time	3 laboratory hours
Submission	Source code / screenshots where applicable, observation, discussion, answers to post-lab tasks

Objectives

- Implement depth-limited search (DLS).
- Explain cutoff caused by a depth limit.
- Implement iterative deepening depth-first search (IDDFS).

Problem Tree

```
S
|-- A
|   |-- C
|   `-- D
|       `-- G
`-- B
    |-- E
    `-- F
```

Step-by-Step Procedure

42. Represent the tree using an adjacency-list dictionary.
43. Write a recursive search that tracks current depth and path.

44. Return failure/cutoff when depth reaches the specified limit before finding the goal.
45. Test DLS with limit 2 and then 3.
46. Implement IDDFS by calling DLS with limits 0, 1, 2, ... until the goal is found.

Complete Solution

```
tree = {
    "S": ["A", "B"],
    "A": ["C", "D"],
    "B": ["E", "F"],
    "C": [], "D": ["G"],
    "E": [], "F": [], "G": []
}

def depth_limited_search(graph, start, goal, limit):
    def rec(node, path, depth):
        if node == goal:
            return path
        if depth == limit:
            return None
        for nbr in graph[node]:
            if nbr not in path:
                result = rec(nbr, path + [nbr], depth + 1)
                if result:
                    return result
        return None
    return rec(start, [start], 0)

def iddfs(graph, start, goal, max_depth=10):
    for limit in range(max_depth + 1):
        result = depth_limited_search(graph, start, goal, limit)
        if result:
            return limit, result
    return None, None

print("DLS limit 2:", depth_limited_search(tree, "S", "G", 2))
print("DLS limit 3:", depth_limited_search(tree, "S", "G", 3))
print("IDDFS:", iddfs(tree, "S", "G", 5))
```

Expected / Sample Output: DLS limit 2: None
DLS limit 3: ['S', 'A', 'D', 'G']
IDDFS: (3, ['S', 'A', 'D', 'G'])

Interpretation

The goal is at depth 3, so a limit of 2 cannot reach it. IDDFS combines the low memory behavior of depth-first search with the ability to discover the shallowest goal by repeatedly increasing the depth bound.

Post-Lab Tasks

- Move G under a different branch and compare the trace.
- Add cycle protection and explain why it matters for general graphs.
- Count how many times node S is revisited by IDDFS and explain the trade-off.

Viva Questions with Short Answers

Question	Short answer
What is the purpose of a depth limit?	To prevent unbounded descent and restrict search to a maximum depth.
Why does IDDFS repeat work?	It restarts DLS with a larger limit, so shallow nodes are revisited.
Why is IDDFS still useful?	The repeated shallow work is often small compared with the number of nodes at the deepest explored level, while memory remains low.

EXPERIMENT 6

Heuristic Functions and A* Search

Item	Details
CLO mapping	CLO2
Recommended time	3 laboratory hours
Submission	Source code / screenshots where applicable, observation, discussion, answers to post-lab tasks

Objectives

- Define heuristic function h(n).
- Distinguish path cost g(n) and estimated total cost f(n)=g(n)+h(n).
- Implement A* search with a priority queue.
- Interpret admissibility and search optimality at an introductory level.

Weighted Search Problem

Edge	Cost
A-B	1
A-C	4
B-D	2
B-E	5
C-E	1
D-G	6
E-G	3

Node	h(n)
A	7
B	6
C	3
D	6
E	2
G	0

Step-by-Step Procedure

47. Represent each adjacency as (neighbor, step_cost).
48. Maintain a priority queue ordered by f = g + h.
49. Store the best g-cost currently known for each node.
50. Pop the lowest-f state. If it is the goal, return its path and cost.
51. For each neighbor, calculate a new g-cost. Update only if it improves the best known cost.
52. Push the updated neighbor with priority new_g + h(neighbor).

Complete Solution

```
import heapq

graph = {
    "A": [("B",1), ("C",4)],
    "B": [("D",2), ("E",5)],
    "C": [("E",1)],
    "D": [("G",6)],
    "E": [("G",3)],
    "G": []
}

h = {"A":7, "B":6, "C":3, "D":6, "E":2, "G":0}

def astar(graph, start, goal, h):
    pq = [(h[start], 0, start, [start])]
    best_g = {start: 0}

    while pq:
        f, g, node, path = heapq.heappop(pq)
        if node == goal:
            return path, g

        for nbr, cost in graph[node]:
            new_g = g + cost
            if new_g < best_g.get(nbr, float("inf")):
                best_g[nbr] = new_g
                new_f = new_g + h[nbr]
                heapq.heappush(pq, (new_f, new_g, nbr, path + [nbr]))

    return None, float("inf")

path, cost = astar(graph, "A", "G", h)
print("Path:", path)
print("Total cost:", cost)
```

Expected / Sample Output: Path: ['A', 'C', 'E', 'G']
Total cost: 8

Solution Explanation

A* combines the exact cost already paid, g(n), with the estimated remaining cost, h(n). In this example the optimal route is A-C-E-G with cost 4+1+3=8. With an admissible heuristic and standard graph-search conditions, A* can guarantee an optimal solution.

Post-Lab Tasks

- Set every h(n)=0. Which familiar algorithm does A* behave like for nonnegative edge costs?
- Make h(B) extremely small and observe the expansion order. Does the final answer necessarily change?
- Design a heuristic for a 2D grid using Manhattan distance.

Viva Questions with Short Answers

Question	Short answer
What is g(n)?	The known path cost from the start node to n.
What is h(n)?	An estimate of the remaining cost from n to a goal.
What is f(n)?	The A* priority: f(n)=g(n)+h(n).
What is an admissible heuristic?	A heuristic that never overestimates the true minimum remaining cost.

EXPERIMENT 7

Introduction to WEKA for AI/ML Experiments

Item	Details
CLO mapping	CLO4 (tool preparation)
Recommended time	3 laboratory hours
Submission	Source code / screenshots where applicable, observation, discussion, answers to post-lab tasks

Objectives

- Launch WEKA Explorer and identify its main tabs.
- Load and inspect a dataset.
- Set a class attribute and run a baseline classifier.
- Interpret the main sections of WEKA result output.

WEKA Explorer Overview

Tab	Purpose
Preprocess	Load data, inspect attributes, apply filters.
Classify	Train/evaluate classifiers such as J48, IBk, NaiveBayes, etc.
Cluster	Run clustering methods such as SimpleKMeans.
Select attributes	Perform feature/attribute selection.
Visualize	Inspect two-dimensional attribute plots.

Step-by-Step Procedure Using Iris

53. Open WEKA and click Explorer.
54. Choose the Preprocess tab and click Open file.
55. Open the sample iris.arff dataset. If the installation stores sample data separately, browse to the WEKA data folder.
56. Confirm that the dataset contains 150 instances and that the final attribute is the class with three species values.
57. Click several attributes to inspect minimum, maximum, mean, standard deviation, and the histogram.
58. Go to Classify. Click Choose -> trees -> J48.
59. Under Test options choose Cross-validation and keep folds = 10. Keep the class attribute as the final attribute.
60. Click Start. Wait for the result buffer to appear.
61. Record correctly/incorrectly classified instances, accuracy, Kappa, per-class statistics, and confusion matrix.
62. Right-click the completed result in the Result list and use Visualize classifier errors if available.
63. Return to Classify and repeat with lazy -> IBk to compare a k-nearest-neighbor classifier.
64. Save or copy the result buffer for the lab report.

Expected Interpretation

With the standard Iris dataset, both J48 and k-NN should normally achieve high classification accuracy under 10-fold cross-validation. Exact figures can vary with WEKA version, algorithm defaults, and randomization settings. The required result is therefore the student's actual WEKA output plus interpretation, not a memorized percentage.

What to Record

Item	Record in lab report
Dataset	File name, number of instances, number/type of attributes, class values.
Algorithm	Exact classifier name and important parameters.
Evaluation	10-fold cross-validation (and seed if changed).
Metrics	Accuracy, confusion matrix, selected per-class metrics.

Item	Record in lab report
Comparison	Which classifier performed better and one reason why the result may differ.

Post-Lab Tasks

- Change IBk from default k to k=3 and k=5. Compare accuracy.
- Apply a normalization/standardization filter and repeat k-NN. Record whether results change.
- Use the Cluster tab to locate SimpleKMeans and identify the parameter that controls number of clusters.

Viva Questions with Short Answers

Question	Short answer
What is ARFF?	WEKA's Attribute-Relation File Format for defining attributes and data instances.
Why use cross-validation?	To estimate generalization performance using repeated train/validation splits within the available data.
What is a confusion matrix?	A table comparing actual classes with predicted classes.
Which WEKA tab is used for clustering?	Cluster.

Machine-Learning Workflow for Experiments 8-10

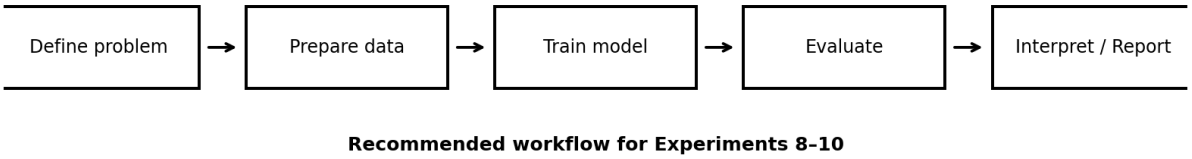


Figure 3. Recommended experimental workflow.

For each machine-learning experiment, record enough information to make the result reproducible: dataset, selected attributes, train/test strategy, random seed, preprocessing, model hyperparameters, evaluation metric, and a short interpretation. A good laboratory answer explains not only the numerical score but what the score means and what limitations remain.

EXPERIMENT 8

Supervised Learning: Linear Regression

Item	Details
CLO mapping	CLO4
Recommended time	3 laboratory hours
Submission	Source code / screenshots where applicable, observation, discussion, answers to post-lab tasks

Objectives

- Explain feature (input) and target (output) variables.
- Fit a linear regression model.
- Interpret slope, intercept, prediction, and R^2.
- Visualize observed points and the fitted line.

Dataset

Study Hours (x)	Score (y)
1	3
2	5
3	7
4	9
5	11
6	13

Model

Simple linear regression models the target as y = b0 + b1 x, where b0 is the intercept and b1 is the slope. The fitted data here follow y = 1 + 2x exactly, which makes the expected result easy to verify.

Step-by-Step Procedure

65. Import NumPy and LinearRegression from scikit-learn.
66. Create X as a two-dimensional array with one input feature and y as a one-dimensional target array.
67. Create LinearRegression() and call fit(X, y).
68. Print the learned slope and intercept.
69. Predict the score for 7 study hours.
70. Calculate R^2 using model.score(X, y).
71. Plot observed data and the fitted line.

Complete Solution

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

X = np.array([[1], [2], [3], [4], [5], [6]])
y = np.array([3, 5, 7, 9, 11, 13])

model = LinearRegression()
model.fit(X, y)

print("Slope:", model.coef_[0])
print("Intercept:", model.intercept_)
print("Prediction for 7 hours:", model.predict([[7]])[0])
print("R^2:", model.score(X, y))

plt.scatter(X[:, 0], y, label="Observed")
plt.plot(X[:, 0], model.predict(X), label="Fitted line")
plt.xlabel("Study hours")
plt.ylabel("Score")
plt.legend()
plt.show()
```

Expected / Sample Output: Slope: approximately 2.0
Intercept: approximately 1.0
Prediction for 7 hours: approximately 15.0
R^2: 1.0

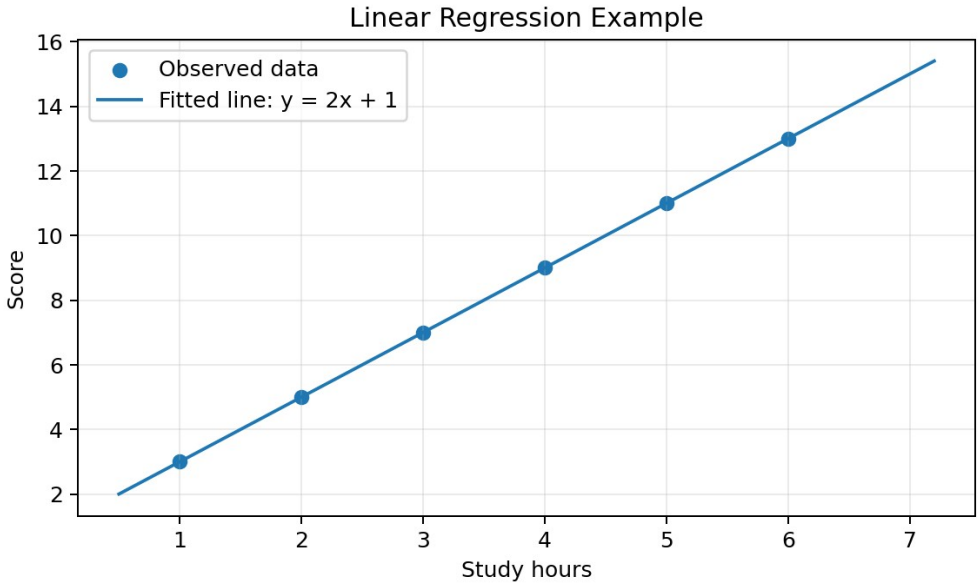


Figure 4. Regression result for the sample data.

Interpretation

The slope 2 means the fitted model associates each additional study hour with an increase of about two score units in this synthetic dataset. The intercept 1 is the fitted value at x=0. R^2=1.0 occurs because every sample lies exactly on one straight line; real data usually produce lower values and require residual analysis.

Post-Lab Tasks

- Change one target value, refit the model, and report the new R^2.
- Split a larger dataset into training and testing sets and report test R^2.
- Explain why correlation alone does not establish causation.

Viva Questions with Short Answers

Question	Short answer
What is a feature?	An input variable used by the model.
What is a target?	The output variable the model is trained to predict.
What does the slope represent?	The fitted change in target for a one-unit increase in the feature.
What does R^2 measure?	The proportion of target variance explained by the model relative to a baseline mean predictor; higher is generally better, but interpretation depends on context.

EXPERIMENT 9

Supervised Learning: k-NN and Decision Tree Classification

Item	Details
CLO mapping	CLO4

Item	Details
Recommended time	4 laboratory hours
Submission	Source code / screenshots where applicable, observation, discussion, answers to post-lab tasks

Objectives

- Train and test two classification algorithms.
- Use a stratified train/test split.
- Compute accuracy and confusion matrices.
- Compare instance-based and tree-based classifiers.

Dataset

Use the built-in Iris dataset: 150 flower samples, four numeric features, and three target classes. The code uses a 70/30 stratified split with random_state=42 for reproducibility.

Step-by-Step Procedure

72. Load the Iris dataset.
73. Split data into training and testing sets with stratify=y.
74. Create KNeighborsClassifier with k=5.
75. Create DecisionTreeClassifier with max_depth=3 and random_state=42.
76. Fit both models on the training set.
77. Predict the test labels.
78. Calculate accuracy and confusion matrices.
79. Compare where the two models make errors.

Complete Solution

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, confusion_matrix

iris = load_iris()
X, y = iris.data, iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.30, random_state=42, stratify=y
)

knn = KNeighborsClassifier(n_neighbors=5)
dtree = DecisionTreeClassifier(max_depth=3, random_state=42)

knn.fit(X_train, y_train)
dtree.fit(X_train, y_train)

pred_knn = knn.predict(X_test)
pred_tree = dtree.predict(X_test)

print("k-NN accuracy:", accuracy_score(y_test, pred_knn))
print(confusion_matrix(y_test, pred_knn))

print("Decision Tree accuracy:", accuracy_score(y_test, pred_tree))
print(confusion_matrix(y_test, pred_tree))
```

Expected / Sample Output: k-NN accuracy: 0.9778 (44/45 correct)
Confusion matrix:
[[15, 0, 0], [0, 15, 0], [0, 1, 14]]

Decision Tree accuracy: 0.9778 (44/45 correct)
Confusion matrix:
[[15, 0, 0], [0, 14, 1], [0, 0, 15]]

Solution Explanation

Both models correctly classify 44 of 45 test samples in this fixed split, but they make different mistakes. k-NN predicts using nearby training samples in feature space; a decision tree predicts by following learned feature-threshold rules. Equal accuracy does not mean the models behave identically.

Important Experimental Note

Feature scaling: k-NN is distance-based, so scaling can matter greatly when features have very different units. Decision trees are generally much less sensitive to monotonic feature scaling.

Post-Lab Tasks

- Evaluate k = 1, 3, 5, 7 and identify the best test accuracy for this fixed split.
- Standardize features using StandardScaler and repeat k-NN.
- Change max_depth for the decision tree and comment on underfitting/overfitting.
- Report precision, recall, and F1-score using classification_report.

Viva Questions with Short Answers

Question	Short answer
What does k mean in k-NN?	The number of neighboring training samples used to vote for the prediction.
Why use stratify=y?	To preserve class proportions approximately across train and test sets.
What is a decision-tree split?	A feature-threshold rule used to divide data into child nodes.
Why is accuracy alone sometimes insufficient?	It can hide class-specific errors, especially when classes are imbalanced.

EXPERIMENT 10

Unsupervised Learning: K-means Clustering

Item	Details
CLO mapping	CLO4

Item	Details
Recommended time	3 laboratory hours
Submission	Source code / screenshots where applicable, observation, discussion, answers to post-lab tasks

Objectives

- Explain clustering and the meaning of k.
- Run K-means on unlabeled feature data.
- Interpret cluster centers, inertia, and silhouette score.
- Visualize the resulting clusters.

Essential Theory

K-means partitions observations into k clusters by iteratively assigning each sample to the nearest centroid and then recomputing centroids as cluster means. The objective is to reduce within-cluster squared distances. Cluster IDs are arbitrary labels and do not automatically correspond to real-world class names.

Step-by-Step Procedure

80. Load the Iris dataset but use only the first two features for easy visualization.
81. Create KMeans with n_clusters=3, random_state=42, and n_init=10.
82. Call fit_predict(X) to obtain cluster labels.
83. Print cluster centers and inertia.
84. Calculate silhouette score.
85. Create a scatter plot colored by cluster label and mark the centroids.
86. Explain why unsupervised cluster IDs should not be interpreted as species labels without additional analysis.

Complete Solution

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

iris = load_iris()
X = iris.data[:, :2]  # sepal length and sepal width

model = KMeans(n_clusters=3, random_state=42, n_init=10)
labels = model.fit_predict(X)

print("Centers:
", model.cluster_centers_)
print("Inertia:", model.inertia_)
print("Silhouette:", silhouette_score(X, labels))

plt.scatter(X[:, 0], X[:, 1], c=labels)
plt.scatter(model.cluster_centers_[:, 0],
        model.cluster_centers_[:, 1],
        marker="X", s=180, edgecolors="black")
plt.xlabel("Sepal length (cm)")
plt.ylabel("Sepal width (cm)")
plt.title("K-means clustering, k=3")
plt.show()
```

Expected / Sample Output: Approximate centers (order may differ by cluster label):
[[6.813, 3.074], [5.774, 2.692], [5.006, 3.428]]
Inertia: approximately 37.051
Silhouette score: approximately 0.445

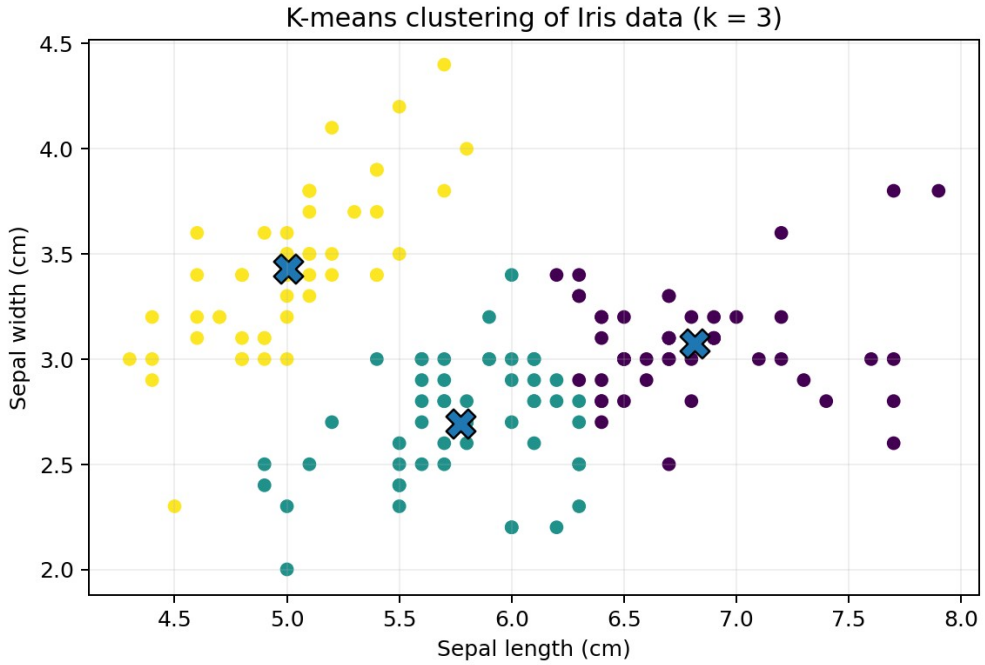


Figure 5. Example K-means result using the first two Iris features.

Interpretation

The silhouette score is positive but not close to 1, indicating only moderate separation when the first two Iris features are used. Inertia always decreases as k increases, so it should not be used alone to select k. A more complete analysis can compare the elbow curve, silhouette score, domain knowledge, and stability across initializations.

Post-Lab Tasks

- Run k=2, 3, 4, and 5. Record inertia and silhouette score.
- Standardize the features before clustering and compare results.
- Use all four Iris features for fitting, then visualize any two features.
- Explain why cluster number 0 is not the same thing as class label 0.

Viva Questions with Short Answers

Question	Short answer
Is K-means supervised?	No. It does not use target labels during fitting.
What is a centroid?	The mean feature vector representing a cluster center.
What is inertia?	The sum of squared distances of samples to their assigned cluster centers.
What does a high silhouette score indicate?	Samples are, on average, well matched to their own cluster and separated from other clusters.

Mini-Project and Project Presentation Guidelines

The syllabus includes a project presentation before the final examination. The project is not counted as an additional experiment in this manual; it is a separate integrative activity using concepts from one or more experiments.

Recommended Project Scope

- Use a small, ethically appropriate dataset that can be processed on ordinary laboratory computers.
- State one clear problem: prediction, classification, clustering, or search/problem-solving.
- Implement at least one baseline method and one improved/alternative method where feasible.
- Use a transparent evaluation plan and report limitations.
- Maintain a reproducible notebook or source-code folder.

Suggested Project Workflow

Stage	Required output
1. Problem definition	One-paragraph problem statement and success criterion.
2. Data / state representation	Dataset description or search state-space description.
3. Method	Algorithm(s), parameters, preprocessing, and implementation details.
4. Experiment	Reproducible run with metrics/trace.
5. Analysis	What worked, what failed, limitations, and possible improvement.
6. Presentation	5-8 slides plus a short live/demo or result walkthrough.

Suggested Presentation Rubric

Criterion	Weight
Problem clarity and motivation	15%
Technical implementation	30%
Evaluation and interpretation	25%
Reproducibility / organization	15%
Presentation and response to questions	15%

Academic integrity: Students must identify datasets, external code, libraries, and any AI-assisted content used. They remain responsible for understanding and being able to explain the submitted work.

Assessment and Record-Keeping Guidance

The supplied syllabus shows continuous internal evaluation components and an overall assessment summary that includes attendance/participation, quiz/presentation, in-course/tutorial, and final examination. This manual does not replace those official components. Instructors may use the experiment record rubric earlier in this manual to grade individual laboratory work while keeping official course percentages unchanged.

What Students Should Keep for the Final Lab Examination

- All ten completed experiment records.
- Working source code for Experiments 1-6 and 8-10.
- WEKA screenshots/result buffers for Experiment 7.
- A one-page comparison sheet for BFS/DFS/Greedy/A*.
- A one-page ML metric sheet: accuracy, confusion matrix, R^2, inertia, silhouette.
- Mini-project slides/code if the course instructor requires them in the examination.

Appendix A - Common Troubleshooting

Problem	Likely cause / remedy
ModuleNotFoundError: sklearn	Install scikit-learn using: python -m pip install scikit-learn. Make sure pip belongs to the same Python interpreter used to run the file.
Matplotlib window does not appear	In notebooks use an inline backend; in scripts make sure plt.show() is called.
Different classification accuracy	Check random_state, stratify option, test_size, library version, and model parameters. Small differences can be valid if configuration differs.
K-means centers appear in a different order	Cluster labels are arbitrary. Compare center coordinates, not cluster ID numbers.
WEKA cannot open	Check Java installation/compatibility and use the WEKA launcher provided by the installation.
WEKA has no class selected	In Preprocess/Classify select the appropriate class attribute, usually the last attribute for Iris.
Search program loops forever	Use a visited set for graph search and avoid revisiting states unless a better path-cost policy requires it.

Appendix B - Blank Observation Template

Field	Student entry
Experiment no. / title	
Date / group	
Input / dataset / graph	

Field	Student entry
Parameters / settings	
Key output / metric	
Observed issue or error	
Interpretation	
Instructor remarks	

Appendix C - Quick Viva Revision

Concept	One-line recall
Agent	Perceives environment and acts.
PEAS	Performance, Environment, Actuators, Sensors.
Forward chaining	Fact-driven rule inference.
Predicate	Property/relation in FOL.
BFS	Shallowest-first using a queue.
DFS	Deep-first using a stack/recursion.
Greedy best-first	Prioritizes h(n).
A*	Prioritizes g(n)+h(n).
Regression	Predicts a numeric target.
Classification	Predicts a categorical label.
k-NN	Predicts from nearby labeled samples.
Decision tree	Predicts through learned feature-threshold rules.
Clustering	Groups unlabeled samples.
K-means	Iterative centroid-based clustering.
Confusion matrix	Actual-versus-predicted class counts.
R^2	Regression goodness-of-fit measure relative to mean baseline.
Silhouette	Cluster cohesion/separation score.

References and Further Reading

- Stuart Russell and Peter Norvig, Artificial Intelligence: A Modern Approach.
- Ian H. Witten, Eibe Frank, Mark A. Hall, and Christopher J. Pal, Data Mining: Practical Machine Learning Tools and Techniques.
- Python documentation - language and standard library reference.
- scikit-learn documentation - supervised and unsupervised learning APIs used in this manual.
- WEKA documentation and sample datasets distributed with WEKA.